ESCUELA POLITÉCNICA
SUPERIOR DE CÓRDOBA
Universidad de Córdoba

Introducción al Aprendizaje Automático

Grado en Ingeniería Informática

Cuaderno de Laboratorio - Práctica 3: Metodología de Evaluación y Rigor Científico

Autores:

Alicia Muriel Fernández
Lucía Cañero Moslero

Octubre, 2026

UNIVERSIDAD
D
CÓRDOBA

Índice general

Índice de figuras **2**

1. Cuaderno de Laboratorio - Práctica 3: Metodología de Evaluación y Rigor Científico **3**
1. Tarea 1 - La lotería de la partición aleatoria 3
   1.1. Registro de resultados (5 particiones aleatorias) 4
2. Tarea 2 - Partición estratificada 5
   2.1. Registro por folds 5
3. Tarea 3 - Comparativa de la varianza y comparación de modelos . . . 7
   3.1. Resumen de resultados 7
   3.2. Gráficas de estabilidad 7
   3.3. Comparación entre modelos: Regresión Logística vs Random Forest 8
   3.4. Distribución de positivos en train y test 9
   3.5. Interpretación 9
4. Tarea 4 - Detección de data leakage 10
   4.1. Comparativa 10
5. Reto - Auditoría metodológica 12
6. Conclusión final 13

Bibliografía **14**

Alicia Muriel Fernández
Lucía Cañero Moslero

Índice de figuras

1. Positivos en test en 5 particiones aleatorias. 4
2. Proporción constante = 0.02 (2 %) utilizando StratifiedKFold 6
3. Comparativa de desviación típica en Accuracy y F1 entre ambos métodos. std = 0.0000 (Varianza nula) 7
4. Comparativa de accuracy entre Regresión Logística y Random Forest. 8
5. Comparativa de F1-score entre Regresión Logística y Random Forest. F1 = 0.00 (Rendimiento nulo) 8
6. Comparación del número de positivos en train y test (StratifiedKFold). 9
7. Comparación de la Accuracy del modelo con y sin la variable trampa 10
8. Correlación de cada variable con la clase objetivo, destacando la anomalía de la variable ID_Hospital_Filtro (variable trampa) como indicio de fuga de información. 11

Alicia Muriel Fernández
Lucía Cañero Moslero

Capítulo 1

Cuaderno de Laboratorio - Práctica 3: Metodología de Evaluación y Rigor Científico

Objetivo

Aprender a realizar evaluaciones honestas en condiciones difíciles, especialmente con datos desbalanceados, y detectar errores metodológicos graves como la fuga de datos (*data leakage*).

Material de partida

Se parte de:

- Un dataset sintético de **1000 pacientes** con **5 variables predictoras**.
- Una clase positiva muy minoritaria (**2 %** de los casos).

1. Tarea 1 - La lotería de la partición aleatoria

Configuración común:
En esta tarea no se entrena ningún modelo de clasificación. El objetivo es analizar el efecto del azar en la partición aleatoria de los datos y estudiar cómo varía la distribución de la clase minoritaria en el conjunto de test. Para ello, se realizan cinco particiones train/test con distintas semillas y se contabiliza el número de ejemplos positivos en cada ejecución.

Alicia Muriel Fernández
Lucía Cañero Moslero

Hipótesis inicial (antes de ejecutar): ¿Crees que el número de positivos en test será estable? ¿Por qué?

Creemos que no será estable, ya que nos encontramos ante un problema con un gran desequilibrio de clases (solo el 2 % de los casos pertenecen a la clase positiva). Al hacer la partición aleatoria con 80 % de entrenamiento y 20 % de prueba, esperaríamos aproximadamente 4 casos positivos, es decir, el 20 % de los 20 pacientes. Sin embargo, al ser aleatorio podría dejar muy pocos positivos en test.

1.1. Registro de resultados (5 particiones aleatorias)

Ejecución	Positivos en test
1	4
2	3
3	5
4	4
5	3

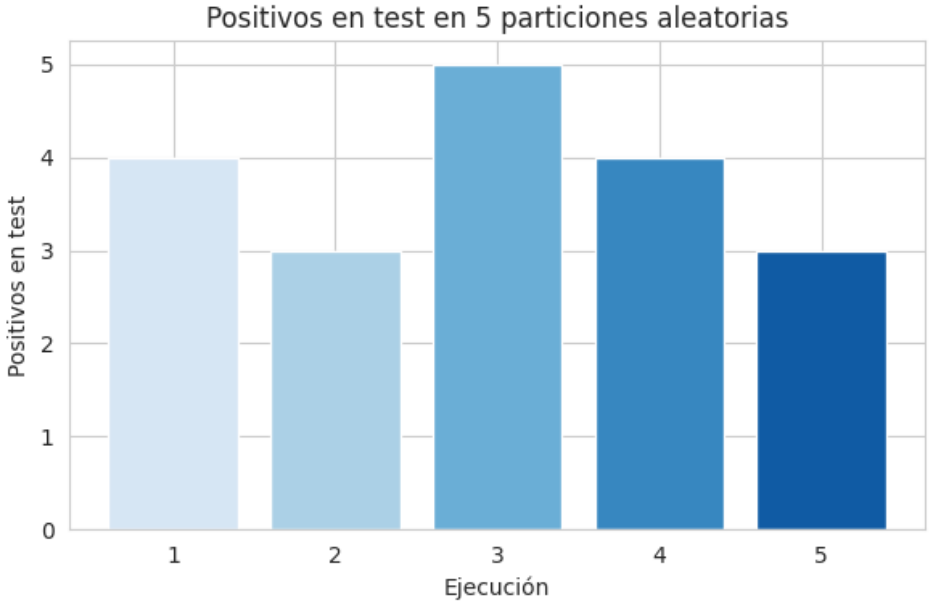


Figura 1: Positivos en test en 5 particiones aleatorias.

Alicia Muriel Fernández
Lucía Cañero Moslero

Análisis:

- **¿Ha cambiado mucho el número de positivos en test entre ejecuciones?**
  Sí, podemos ver que hay variabilidad en los resultados ya que el valor está entre 3 y 5, cerca del valor 4 que esperábamos. Aunque la diferencia parece pequeña, pasar de 3 a 5 implica casi duplicar la representación de la clase minoritaria de una ejecución a otra.

- **¿Sería fiable la evaluación si el azar dejara solo 0 o 1 positivos en test?**
  No sería fiable. Con 0 positivos, es imposible evaluar la sensibilidad del modelo ante la clase minoritaria. Con 1 solo positivo, las métricas se vuelven bastante inestables: un único acierto o fallo dispararía el rendimiento al 100 % o lo hundiría al 0 %, llevando a conclusiones engañosas. Por ejemplo, si el modelo acierta este único caso obtendríamos el 100 % de sensibilidad, pero si lo falla tendríamos un 0 %.

2. Tarea 2 - Partición estratificada

2.1. Registro por folds

Fold	Positivos en test	Tamaño del fold	Proporción clase 1
1	4	200	0.02
2	4	200	0.02
3	4	200	0.02
4	4	200	0.02
5	4	200	0.02

Alicia Muriel Fernández
Lucía Cañero Moslero

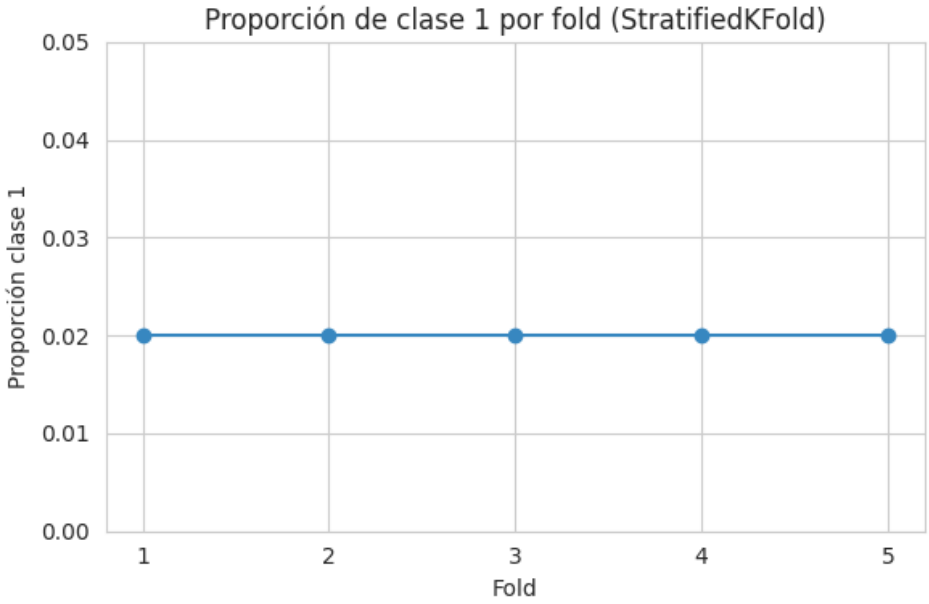


Figura 2: Proporción constante = 0.02 (2 %) utilizando StratifiedKFold

Comprobación de la proporción:
Podemos observar (en la tabla anterior "Registro por folds") que la proporción de la clase minoritaria se mantiene constante. El dataset original está formado por 1000 pacientes, de los cuales el 2 % pertenece a la clase positiva (clase 1), es decir, 20 casos positivos. Al dividir los datos en 5 particiones, cada conjunto de test contiene 200 muestras. Debido a la estratificación conseguimos que el algoritmo reparta equitativamente los 20 positivos entre los 5 folds, asignando 4 positivos por fold: 4/200 = 0.02. Con esto demostramos que StratifiedKFold cumple su función de mantener una distribución equitativa de clases en todas las particiones.

Análisis: Explica por qué `StratifiedKFold` es más adecuado que una partición aleatoria simple en este problema.
En problemas como este con gran desbalanceo, si usamos una partición aleatoria como en la tarea 1, no garantizamos que se respete la proporción de clases ya que es azar puro. Como ya hemos explicado en la tarea anterior, esta situación provoca métricas inestables o falsas. La solución correcta, como podemos observar en esta tarea, es utilizar StratifiedKFold porque obliga a que cada partición tenga la misma proporción de clases que el dataset original. Aunque esta técnica no mejora el poder predictivo del modelo, garantiza una evaluación más estable. Al asegurar que siempre haya ejemplos positivos para evaluar, la varianza de las métricas disminuye y los resultados son más fiables.

3. Tarea 3 - Comparativa de la varianza y comparación de modelos

3.1. Resumen de resultados

Tras eliminar la variable trampa del conjunto de datos, evaluamos dos modelos de clasificación (Regresión Logística y Random Forest) realizando 10 iteraciones para los métodos de validación KFold y StratifiedKFold. En ambos casos, la *accuracy* se mantiene alrededor del 98 % y el F1-score es igual a 0. Esto se debe al fuerte desbalanceo del dataset: la clase positiva representa únicamente el 2 % de las instancias, por lo que un modelo que predice siempre la clase mayoritaria ya obtiene una accuracy muy elevada.

A continuación se muestran los resultados obtenidos:

Método	Accuracy media	std(Accuracy)	F1 media	std(F1)
KFold	0.9799	0.0126	0.00	0.00
StratifiedKFold	0.98	0.0000	0.00	0.00

3.2. Gráficas de estabilidad

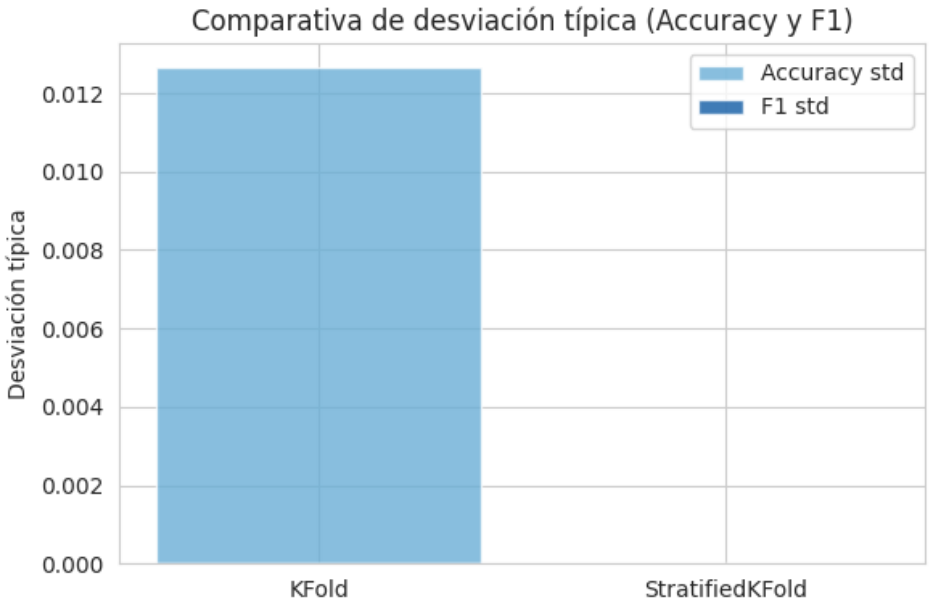


Figura 3: Comparativa de desviación típica en Accuracy y F1 entre ambos métodos. std = 0.0000 (Varianza nula)

Alicia Muriel Fernández
Lucía Cañero Moslero

3.3. Comparación entre modelos: Regresión Logística vs Random Forest

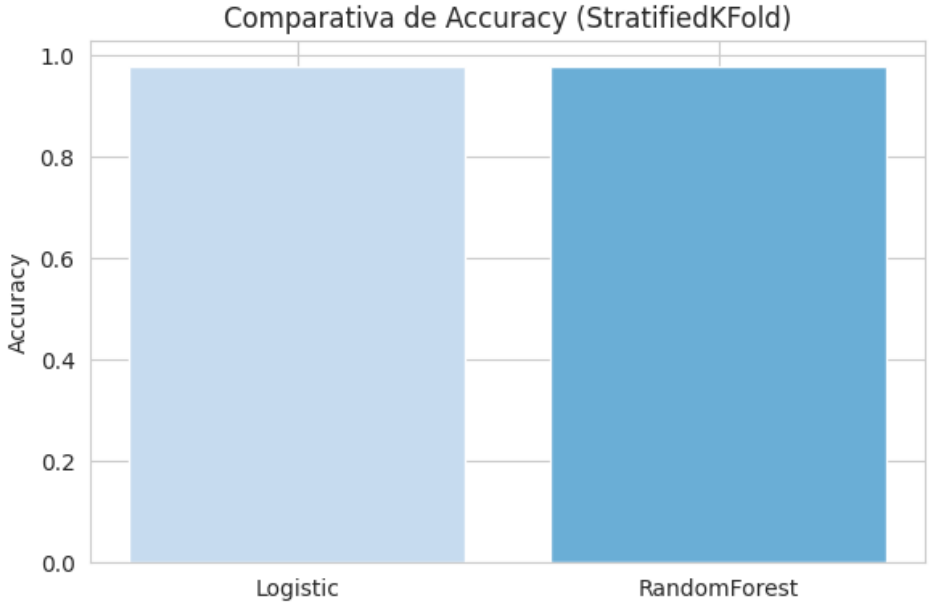


Figura 4: Comparativa de accuracy entre Regresión Logística y Random Forest.

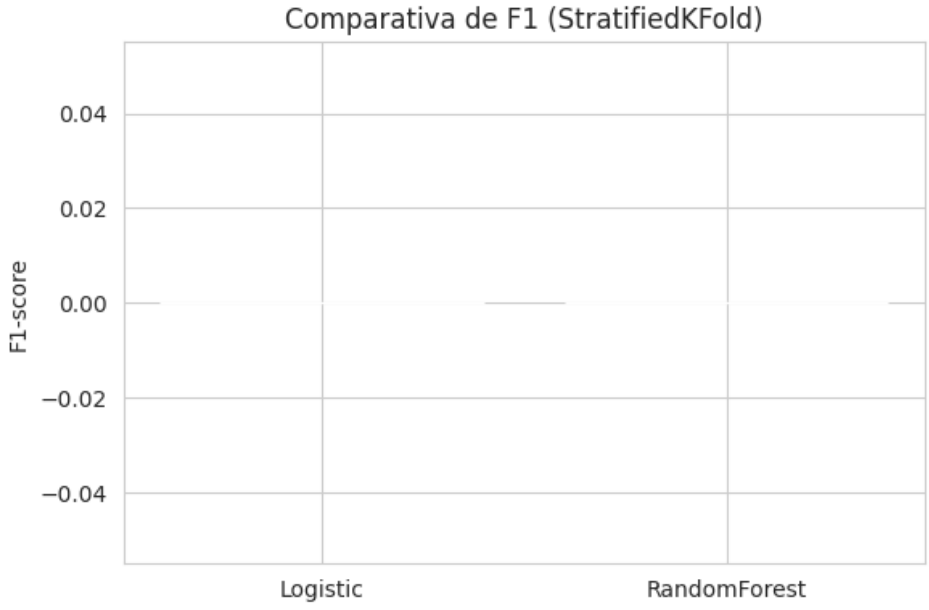


Figura 5: Comparativa de F1-score entre Regresión Logística y Random Forest.
F1 = 0.00 (Rendimiento nulo)

3.4. Distribución de positivos en train y test

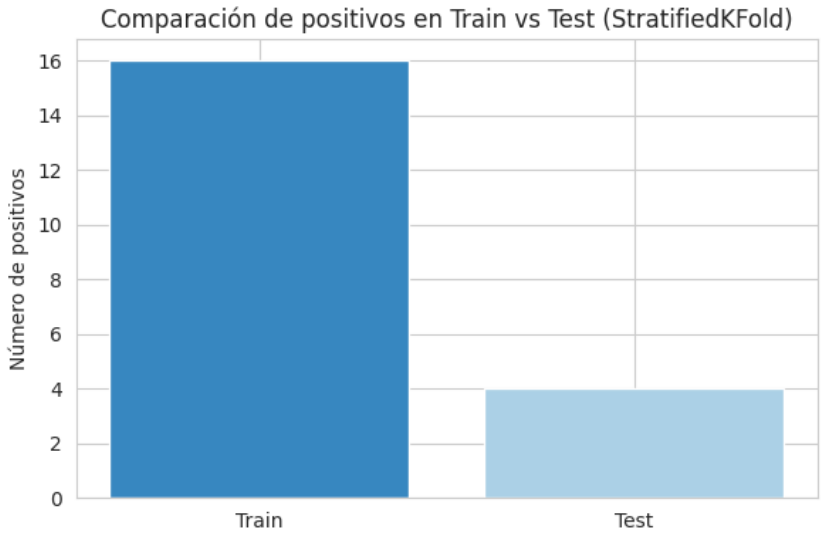


Figura 6: Comparación del número de positivos en train y test (StratifiedKFold).

3.5. Interpretación

Las gráficas muestran claramente que **StratifiedKFold es el método más estable**, el que depende menos del azar a la hora de particionar un dataset tan desbalanceado y, por tanto, el método de validación que inspira más confianza. En un problema tan desiquilibrado, una partición aleatoria puede generar folds con muy pocos positivos, lo que provoca una gran variabilidad en las métricas. La estratificación evita este problema al mantener fija la proporción de clases en cada fold, reduciendo la varianza casi a cero.

La comparación entre modelos revela que tanto Regresión Logística como Random Forest obtienen exactamente los mismos resultados. Esto demuestra que el dataset no contiene información suficiente para aprender a detectar la clase minoritaria: ambos modelos terminan prediciendo siempre la clase mayoritaria, lo que explica la accuracy alta y el F1-score igual a 0. Esto es la prueba que una Accuracy alta puede no significar que el modelo detecte bien la clase minoritaria.

Finalmente, la gráfica de distribución de positivos confirma visualmente que StratifiedKFold reparte las clases de forma perfectamente equilibrada en cada fold. Esta estabilidad es esencial para obtener una evaluación fiable en problemas desbalanceados.

En resumen, la tarea muestra que **la estratificación no mejora el modelo, mejora la calidad de la evaluación.**

4. Tarea 4 - Detección de data leakage

4.1. Comparativa

Escenario	Accuracy	F1
Sin variable trampa	0.98	0.00
Con variable trampa	0.997	0.914

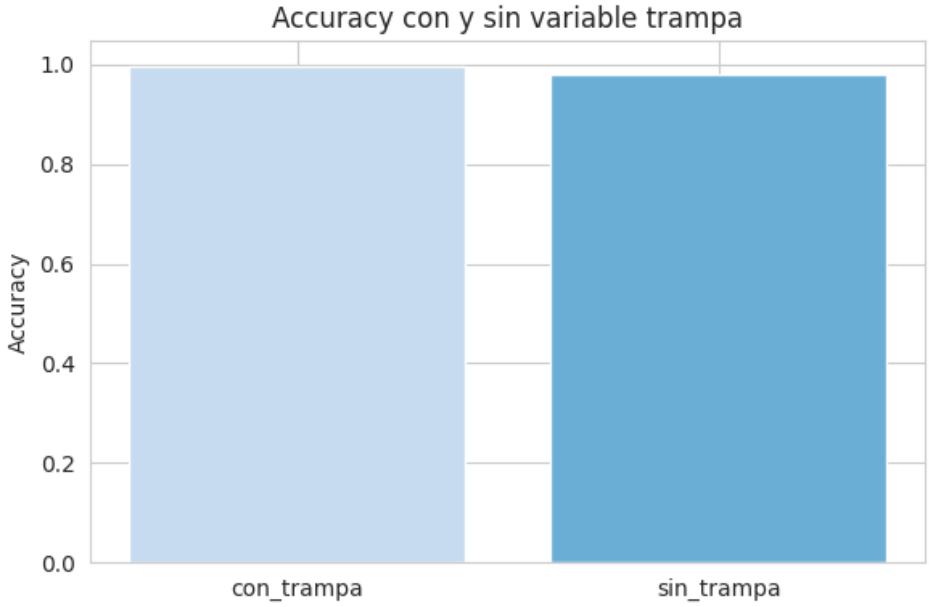


Figura 7: Comparación de la Accuracy del modelo con y sin la variable trampa

Alicia Muriel Fernández
Lucía Cañero Moslero

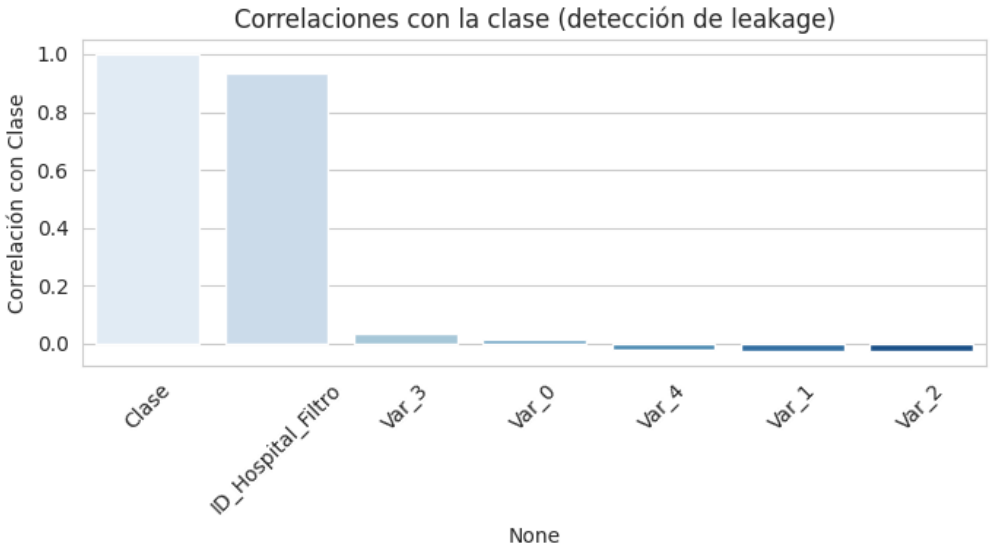


Figura 8: Correlación de cada variable con la clase objetivo, destacando la anomalía de la variable ID_Hospital_Filtro (variable trampa) como indicio de fuga de información.

Preguntas:

- ¿Qué variable sospechas que está filtrando información del objetivo?
- ¿Por qué esa variable invalida la evaluación?
- ¿Qué diferencia conceptual hay entre un modelo "bueno" y un modelo contaminado por leakage?

Interpretación:

¿Qué variable sospechas que está filtrando información del objetivo?

Al mirar la gráfica de correlaciones, se ve claramente que la variable `ID_Hospital_Filtro` tiene una relación muy alta con la clase, mucho más que el resto. Esa diferencia tan grande hace pensar que esta variable está filtrando información del objetivo. Además, cuando se entrena el modelo con ella, la Accuracy sube casi al 100 %, algo que no es nada realista y que confirma que está actuando como una variable trampa.

¿Por qué esa variable invalida la evaluación?

Esta variable afecta a la evaluación porque hace que haya *leakage*, es decir, el modelo está usando información que no debería tener en el momento de predecir. En vez de aprender patrones reales, simplemente aprovecha esa pista directa sobre la etiqueta y por eso las métricas salen tan altas. Cuando se elimina la variable trampa, la Accuracy baja un poco y el F1 cae a 0.00, lo que demuestra que el modelo no sabía detectar la clase minoritaria: solo estaba copiando la información filtrada.

¿Qué diferencia conceptual hay entre un modelo "bueno" y un modelo contaminado por leakage?
Un modelo "bueno" aprende relaciones reales entre las variables y la clase, y es capaz de mantener un rendimiento razonable cuando se enfrenta a datos nuevos. Un modelo con leakage, en cambio, solo parece bueno porque ha visto información que no debería. Sus métricas engañan: en validación funciona muy bien, pero en un caso real fallaría porque esa información filtrada no estaría disponible. En resumen, no es un modelo mejor, sino un modelo que está "haciendo trampa".

En definitiva, esta tarea muestra que un modelo puede parecer muy bueno si se cuela información que no debería, pero ese rendimiento no es real. Por eso es tan importante revisar las variables y asegurarse de que no haya filtraciones antes de confiar en las métricas.

5. Reto - Auditoría metodológica

En problemas desbalanceados, repartir los datos al azar puede ser arriesgado porque algunos conjuntos pueden quedarse casi sin ejemplos de la clase minoritaria. Cuando eso pasa, el modelo no se evalúa en buenas condiciones y las métricas pueden cambiar mucho según la partición que haya tocado. La validación estratificada evita este problema manteniendo la misma proporción de clases en cada fold, lo que hace que los resultados sean más estables y menos dependientes del azar.

Otro aspecto importante es el *data leakage*. Si el modelo recibe información que no debería conocer, los valores de las métricas aumentan y parece que el modelo funciona muy bien, cuando en realidad no ha aprendido nada útil. En esos casos, las conclusiones del experimento dejan de tener sentido porque ese rendimiento no aparecería cuando el modelo se enfrente a datos reales.

Para evitar estos problemas, es fundamental revisar bien las variables antes de entrenar, usar validación estratificada cuando haya desbalanceo, separar correctamente entrenamiento y test y asegurarse de que no haya ninguna variable que esté filtrando la etiqueta. También ayuda repetir los experimentos varias veces para comprobar que los resultados son consistentes.

6. Conclusión final

De esta práctica hemos aparendido lo importante que es trabajar con una buena metodología, sobre todo cuando tratamos con problemas de clasificación muy desbalanceados. Hemos visto que fijarse solo en la accuracy puede llevarnos a conclusiones equivocadas: un modelo puede acertar mucho simplemente porque casi todos los datos pertenecen a la clase mayoritaria, aunque en realidad no esté aprendiendo nada útil sobre la clase minoritaria.

También hemos observado que hacer particiones aleatorias sin más puede generar resultados engañosos. Para evaluar un modelo de forma robusta, es fundamental usar técnicas como la validación estratificada (StratifiedKFold), que mantiene la proporción real de clases en cada fold. Esto reduce muchísimo la variabilidad entre ejecuciones y nos da una evaluación mucho más estable y realista, es decir, la estratificación no hace que el modelo sea mejor, pero sí hace que la evaluación sea real.

Además, la detección de la variable trampa nos ha servido para entender qué es el data leakage. Cuando un modelo recibe información que no debería tener en un escenario real, su rendimiento se dispara de forma sospechosa. Y esto invalida por completo cualquier conclusión que pudiéramos sacar.

En definitiva, una métrica alta no garantiza que un modelo esté funcionando bien. Lo que realmente importa es la metodología: cómo se preparan los datos, cómo se plantea la validación y cómo se interpretan los resultados.

Alicia Muriel Fernández
Lucía Cañero Moslero

Bibliografía

[1] Sebastián Ventura and Christian Luna. Práctica 3 - evaluación honesta y rigor experimental (diapositivas), 2026. Asignatura: Introducción al Aprendizaje Automático. Accedido en marzo 2026.

[2] Sebastián Ventura Soto. Tema 04 - ajuste fino y selección de modelos (diapositivas de teoría), 2026. Asignatura: Introducción al Aprendizaje Automático. Accedido en marzo 2026.

[3] Sebastián Ventura Soto. Tema 07 - evaluación de clasificadores (diapositivas de teoría), 2026. Asignatura: Introducción al Aprendizaje Automático. Accedido en marzo 2026.

[4] Sebastián Ventura Soto. Tema 05b - clasificación logística. (diapositivas de teoría), 2026. Asignatura: Introducción al Aprendizaje Automático. Accedido en marzo 2026.

[5] Universidad de Córdoba. Guion práctica 3: Metodología de evaluación y rigor científico, 2026. Asignatura: Introducción al Aprendizaje Automático (3ª Ingeniería Informática). Accedido en marzo 2026.